# A Practical and Fully Distributed E-Voting Protocol for the Swiss Context

Véronique Cortier
*Université de Lorraine, CNRS, Inria*

Alexandre Debant
*Université de Lorraine, CNRS, Inria*

Olivier Esseiva
*Swiss Post*

Pierrick Gaudry
*Université de Lorraine, CNRS, Inria*

Audhild Høgåsen
*Swiss Post*

Chiara Spadafora
*Swiss Post and Università di Trento*

## Abstract

Internet voting in Switzerland for political elections is strongly regulated by the Federal Chancellery (FCh). It puts a great emphasis on the individual verifiability: security against a corrupted voting device is ensured via return codes, sent by postal mail. For a long time, the FCh was accepting to trust an offline component to set up data and in particular the voting material. Today, the FCh aims at removing this strong trust assumption.

We propose a protocol that abides by this new will. At the heart of our system lies a setup phase where several parties create the voting material in a distributed way, while allowing one of the parties to remain offline during the voting phase. A complication arises from the fact that the voting material has to be printed, sent by postal mail, and then used by the voter to perform several operations that are critical for security. Usability constraints are taken into account in our design, both in terms of computation complexity (linear setup and tally) and in terms of user experience (we ask the voter to type a high-entropy string only once).

The security of our scheme is proved in a symbolic setting, using the ProVerif prover, for various corruption scenarios, demonstrating that it fulfills the Chancellery's requirements and sometimes goes slightly beyond them.

## 1 Introduction

Switzerland has a long history in electronic voting, introduced in nationally binding elections in 2004. The Swiss regulation is demanding. In particular, since 2013, the Federal Chancellery requires the two main security properties usually considered in e-voting, namely vote privacy and verifiability [18]. *Vote privacy* guarantees that no one learns how a voter voted. *Verifiability* ensures that the result corresponds to the votes of the Voters. More specifically, verifiability is often split in several sub-properties:

- *Cast-as-intended*: voters can check that the ballot sent by their voting device contains their intended vote.
- *Recorded-as-cast*: voters can check that the system has registered the ballot they have cast.
- *Universal verifiability* (or tallied-as-recorded): the result corresponds to the registered ballots.
- *Eligibility*: registered ballots have been cast by honest voters or by voters under the control of the attacker. This property prevents ballot stuffing, for example addition of ballots on behalf of absentee voters.

Interestingly, Switzerland, like for instance Estonia [24], has a strong emphasis on cast-as-intended and recorded-as-cast. Even if the voting devices are corrupted (*e.g.* controlled by a nation-scale attacker), the system must guarantee that the vote of a voter cannot be modified without detection. According to the Swiss regulations, this property must be achieved through *return codes*. Namely, before the election, voters receive a voting card like the one displayed in Figure 1. Each candidate is associated with a short 4-digit return code. The codes differ for each voter. Once a voter has selected a candidate on their voting device, they expect the device to display the code that matches the one indicated on their voting card for that candidate. The device, even if corrupted, may only obtain the correct return code if it did send the intended candidate in the encrypted ballot. A key challenge of this mechanism is that it should not jeopardize vote privacy. In particular, the system needs to be able to compute the return code without learning the corresponding vote. In what follows, we merge cast-as-intended and recorded-as-intended into a single property, *recorded-as-intended*, since the distinction is not relevant in the Swiss context as there is no public ballot board. Hence what really matters is which ballots will be counted.

In 2015, Scytl [20] proposed a voting protocol with such a mechanism for recorded-as-intended. It was used in elections in several Swiss Cantons. However, due to a serious

This work benefited from funding managed by the French National Research Agency under the France 2030 programme with the reference ANR-22-PECY-0006.

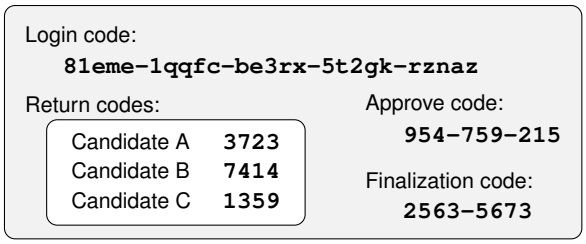


Figure 1: Voting card.

cryptographic flaw discovered in 2019 in zero-knowledge proofs [15], e-voting was temporarily discontinued throughout Switzerland. Swiss Post then fixed the protocol and re-implemented most of it [32]. E-voting was reintroduced with the Swiss Post protocol in several Swiss Cantons in 2023.

*Trust assumptions.* The Swiss Post protocol guarantees vote secrecy and verifiability under the trust assumptions prescribed by the Federal Chancellery [18]. As expected, all online communications are under the control of the attacker. Moreover, as already mentioned, the voting device is not trusted for verifiability (it may attempt to change the vote), but it is trusted for vote privacy, since for usability reasons the voter selects their candidate directly on their device, which therefore learns the vote. The recorded-as-intended mechanism is ensured through 4 online servers (called **Control Components** - CCs), among which only one is trusted for recorded-as-intended and none of them is trusted otherwise, in particular for vote privacy[1]. In contrast, the Swiss Post protocol makes very strong assumptions during the setup. Indeed, the entity that generates the voting material (called **Setup Component** - SC) is fully trusted.

*Limitations.* Assuming a fully honest SC creates an important single point of failure in the Swiss Post protocol. Indeed, if corrupted, this component can break all the desired properties, namely:

- it may add ballots on behalf of absentee voters since it knows the entire voting material;
- it may break vote privacy since the return codes are sent back in clear to voters during the voting phase and the SC knows the association between return codes and candidates;
- it may also modify the vote of the voters, in collusion with their voting devices, since it knows the return codes.

While this complies with the Swiss legal requirements, this gives a lot of power to the entity running the setup.

The Swiss authorities have acknowledged this issue [17] and now aim to reduce the trust placed in the setup, in line with the E-voting Catalogue of Measures by the Confederation and Cantons [19]. As a result, the Swiss Post protocol, the only one currently in use, will no longer be compliant.

In contrast, another protocol, CHVote [22], has been proposed for the Swiss context and it does not require *any* assumption of the SC since the voting material is solely generated by the CCs at the beginning of the protocol. However, this protocol assumes a high trust in the CCs: at least one of them must be trusted for both verifiability and privacy. If they are all dishonest, they may add ballots since they know the voting material. They may also break privacy since they know the return codes. Note that several attacks have been found against CHVote [13], demonstrating *e.g.* that even a single dishonest CC can break verifiability, but we do not discuss them here. The rationale behind CHVote is that the CCs should be operated by distinct entities. In practice, this separation has not been implemented in any elections in Switzerland, as it would require a more complex infrastructure and higher costs. Indeed, operating an online CC is a challenging task. It requires to implement a strict access control (logical and physical), strong DDoS protections, and to manage configurations and updates. In practice, in elections in Switzerland, the CCs are operated by a single company, with distinct administration teams, while the SC is operated by the election authorities (namely, the Cantons). The Cantons do not possess their own infrastructure and cannot be online during the election. Hence the challenge is to split the trust between the SC, operated by the election authorities, and the CCs, operated by a company, in order to preserve both vote privacy and verifiability. Note that an offline SC also strengthens the security of the protocol since an offline component is less subject to attacks.

*Contributions.* We propose a protocol that makes the best of the two approaches, balancing the trust between the SC and the CCs: either the SC or 1 out of 4 CCs is trusted for vote privacy and verifiability. This contrasts with the Swiss Post protocol that always trusts SC and with CHVote that always assumes 1 honest CC out of 4.

The intuition of our protocol is simple: we consider the SC and the 4 CCs as 5 entities that generate the voting material in Multi-Party-Computation (MPC) flavor. A key challenge is that the 5 entities play an asymmetric role. Indeed, SC needs to remain offline during the voting phase, where a return code is sent back to the voter. We therefore need to design an MPC protocol that generates the return codes during the setup and computes them during the voting phase, while one of the 5 entities will be offline. Moreover, this mechanism should preserve vote privacy. To achieve this, we combine mixnet and masking techniques during the setup: the SC masks and shuffles the list of possible votes, so that the CCs only know the correspondence between the return codes and the *masked* votes. Symmetrically, the CCs mask (through an exponentiation) and shuffle the list of possible votes to guarantee vote privacy against a dishonest SC. Then the (offline) SC prepares its share of the return codes in an encrypted manner, to be

[1]The Federal Chancellery allows to trust one out of 4 components for vote privacy, if they are operated by distinct companies, which has not been the case in practice.

sent to the voter on its behalf during the voting phase.

During the voting phase, similarly to the Swiss Post protocol, the voting device (hereafter referred to as the Client) prepares both the encrypted ballot $\mathsf{enc}(v)$ and a masked version $v^k$ of the vote in order to request the corresponding return code. It must also prove in zero-knowledge that the same $v$ was encoded in both parts. Such a proof is however not an instance of the Maurer framework [27] and we found that the zero-knowledge proof provided in Swiss Post protocol (Algorithm 5.4 of [32]) leaks additional data, without proving that it remains zero-knowledge. We therefore reworked this zero-knowledge proof and proved its security. Similarly, the components must provide a proof of correct mixing and exponentiation without revealing the intermediate computations. We provide the corresponding zero-knowledge proofs (arguments, actually). Our protocol remains linear in the number of voters and candidates and does not impact the computational cost for the Client compared to the Swiss Post protocol.

Our protocol complies with the E-voting Catalogue of Measures by the Confederation and Cantons [19], and could therefore serve as a candidate for the next generation e-voting protocol in Switzerland.

We formally analyze our protocol in a symbolic model, using the popular tool ProVerif [6], used to analyze several standard protocols such as TLS 1.3 [5] and Signal [25]. We study various corruption scenarios and we show that:

- vote privacy is preserved as soon as the decryption authorities are honest and either SC or 1 out of 4 CCs is honest;
- universal verifiability is preserved as soon as some honest observer (an auditor) checks the zero-knowledge proofs provided by the different entities;
- eligibility is preserved as soon as either SC or 1 out of 4 CCs is honest;
- recorded-as-intended is preserved as soon as 1 out of 4 CCs is honest. We remark that this stronger trust assumption is needed for any protocol with return codes. Indeed, by design, the Client and the CCs must be able to return the appropriate code for any choice of the voter. Hence if the voter wants to vote for A, the attacker may simulate a voting session with a ballot for A in order to compute the right return code and then restart a new session with a ballot for B. Therefore at least one CC must be honest to preserve cast-as-recorded.

Providing a symbolic proof of security is not only a standard way to gain confidence in the design of a protocol but is actually part of the Swiss requirements [18]. Our symbolic model is the first one that also models the setup phase for the privacy property. This phase was abstracted away in the models provided for the Swiss Post protocol [31] while the CHVote protocol does not provide any proof of privacy [4]. The reason is that proofs of privacy are notoriously more difficult in symbolic models as they require to analyze equivalence properties. Another issue for our symbolic proof is due to the fact that our protocol makes use of mixnets both during the tally and the setup phase. A mixnet takes an arbitrary number of inputs and returns them in a random order, typically after some re-randomization. This cannot be modeled by a function since the number of arguments is not fixed. We therefore had to design a model as faithful as possible but such that it was still possible to prove privacy with the ProVerif tool.

*Related work.* Brelle et al [10] have proposed a recorded-as-intended mechanism based on PET (Plaintext Equivalence Test). As for CHVote, it assumes a high trust in the online CCs: when they are all compromised, they can break both eligibility and vote secrecy. Another idea is to use code voting [14, 21]: instead of entering their vote in the Client, the voter enters a code, where the correspondence between votes and codes is provided on the voting material. Intuitively, each voting code encodes or points to an encrypted ballot containing the vote. Return codes are then used to make sure that the Client does not change the vote (*e.g.* encrypting another vote). The main advantage of this approach is that it guarantees vote privacy even against a dishonest Client. However, in addition to usability issues, the main drawback is that it places full trust in the SC that can now very easily swap votes, without even the help of the Client: it simply needs to write "B" instead of "A" in front of the code corresponding to A, and vice-versa. Worse, this attack can be performed without any cryptographic skills by a neighbor or a roommate by collecting the voting material at home and re-printing a modified version of it.

## 2 High-level View of the Protocol

We first present here a high level view of the protocol, focusing on the view of the Voter.

### 2.1 Participants

- **Voters and their Clients.** Voters vote from home, by connecting to a web site. Beforehand, they have received voting material by postal mail. When studying the recorded-as-intended property, we need to consider a Voter and their Client (computer and software running on it) as separate entities.
- **Control Components.** ($\mathsf{CC}_j$, for $j = 1, \ldots, m$) These are $m$ independent servers (typically, $m = 4$) that participate in the setup phase, and will stay online during the voting phase, in order to send return codes to the Voters.
- **Setup Component.** (SC) This is a server that participates in the setup phase, and will remain offline during the voting phase. In some parts of the setup, SC behaves

similarly to the CCs, and in that case, the associated data is indexed by $j = 0$, as if SC were a $\mathsf{CC}_0$ component.

- **Printer.** At the end of the setup phase, this entity receives data to be printed and sent to the Voters by postal mail.
- **Talliers.** These are servers in charge of generating the public key of the election and storing the decryption key in a distributed manner. They participate in the tally phase, after the voting phase.
- **Auditors.** Throughout the protocol, audit data is collected, including public data, signatures, and zero-knowledge proofs, so that verification by independent auditors provides additional security properties.

We abstract away an entity that is in charge of preparing the list of legitimate Voters, together with their postal addresses (so that the Printer can send the voting material), the list of questions and voting options for each question, and a unique election identifier. We also assume that public keys of all the parties are known by everyone.

In the presentation, we view the Talliers as different entities. We will see in the security analysis that with the trust assumptions defined by the Federal Chancellery, we do not lose any security property by having the set of Talliers formed by the Setup and the Control Components.

## 2.2 Overview of the Phases

**Setup phase.** A key generation protocol is run by the Talliers to set the main public key of the election $\mathsf{pk}_{EL}$. This is done in a standard way, with or without a threshold.

In parallel of the key generation, the setup protocol is run by the Setup and the Control Components in order to produce two pieces of data:

- **Data for the Printer.** At the end of the setup phase, the Printer receives (encrypted with its public key), for each Voter, the information to be printed on a voting card (as in Figure 1) and sent to them by postal mail.
- **Data for each Control Component.** During the setup phase, each CC will remember some information that will be required during the voting phase. In particular, for each Voter, they get a list of partial return codes, associated to a list of masked voting options.

The transcript of all the communications involved during the setup phase is recorded for auditing purposes.

At the end of the setup phase, a delay of several days (or even weeks, in the case where Voters are living abroad) must be set, so that Voters have time to receive the voting material from the Printer.

**Voting phase.** First, a Voter connects to their Client and enters their login code (LC), read on the their card. The Client prompts the possible choices, and the Voter selects one of them, denoted by $v_i$. The Client prepares a ballot that contains $\{v_i\}_{\mathsf{pk}_{EL}}$, an encryption of $v_i$ with the main election key $\mathsf{pk}_{EL}$, and a masked version of $v_i$. This is sent to all the CCs.

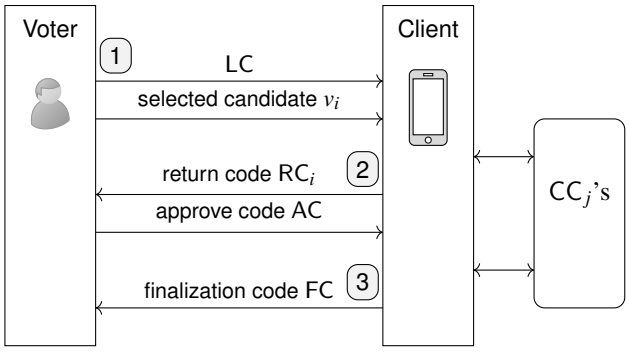


Figure 2: Voter's view of the voting phase. Step 1: authentication and candidate selection; Step 2: check validity and confirmation; Step 3: final check.

From the masked version, each $CC_j$ retrieves in its local data the corresponding partial return code $\mathsf{pRC}_{j,i}$ that is sent back. The Client also receives via the CCs a partial return code generated by the SC, encrypted with a key that is unknown to the CCs. The Client recombines all of these into a return code $\mathsf{RC}_i$ that is shown to the Voter.

The Voter must check that this $\mathsf{RC}_i$ code corresponds to the one printed on their voting card aside the selected candidate $v_i$. If this is not the case, then the Voter must stop the process and complain. If the return code is correct, the Voter retrieves the approve code (AC) from the voting card and enters it to their Client.

The Client encrypts AC with a public key shared by the CCs, who engage in a small protocol based on plaintext equivalence test, to check that this is the correct code.

Finally, if AC is correct, each CC records the ballot for tallying, and sends back a partial finalization code $\mathsf{pFC}_j$, which the Client combines into a finalization code FC, which is shown to the Voter. The Voter must check that FC corresponds to the one written on their voting card, and complain otherwise.

The Voter's view during the voting phase is summarized in Fig. 2.

**Tally phase.** At the end of the voting phase, the CCs agree on the list of ballots to be tallied. The transcript of their communications during the voting phase is also kept for auditing; this contains enough information to resolve a potential disagreement on the list of valid ballots. Then, the Talliers use a decryption mixnet to obtain the result, and publish it.

# 3 Complete protocol description

We consider the following voting context. There is a single question, and Voters can choose one among $n$ candidates, for

answering the question.

## 3.1 Key idea for return codes

The voting card of a Voter must contain a list $(v_i, \mathsf{RC}_i)_{1\le i\le n}$, where $\mathsf{RC}_i$ is a short return code (typically 4 digits) corresponding to candidate $v_i$. This return code $\mathsf{RC}_i$ will be obtained as the XOR of partial return codes $(\mathsf{pRC}_{j,i})_{0\le j\le m}$, sent by the Setup Component (the $j=0$ contribution) and the $m$ Control Components (the $j>0$ contributions).

While the CCs must be able to send the appropriate partial code during the voting phase, they must not learn the choice $v$ of the Voter: this must be sent in an oblivious way. We take an ad-hoc approach, where most of the complexity is concentrated during the setup phase, so that the voting phase is light.

We mask the choice corresponding to a (partial) return code in two steps. On one hand, the set of encrypted $(v_i)_{1\le i\le n}$ is shuffled, where all parties contribute to the permutation, using a verifiable mixnet. The permutation must be transmitted to the Printer, so that it can display the return codes in front of the corresponding candidates. On the other hand, a masking based on exponentiation is also applied, so that the CCs cannot deduce the voting choice from the request of the Voter.

At the end of the setup phase, each $\mathsf{CC}_j$ gets a list of pairs $(v_{\sigma(i)}^{\mathsf{sv_{id}}},\ \mathsf{pRC}_{j,i})_{1\le i\le n}$, where $\sigma$ is a permutation and $\mathsf{sv_{id}}$ is an exponent, both of them being unknown to SC and the CCs. At the same time, the Printer will get the permutation and the return codes, in order to print the list of pairs $(v_i,\ \mathsf{RC}_{\sigma^{-1}(i)})_{1\le i\le n}$ and transmit this to the Voter. As for the $\mathsf{sv_{id}}$, it is transmitted to the Voter partly via the voting card, and partly by the CCs during the voting phase.

The role of the setup phase is also to prepare the part where the Voter sends their approve code (if they are happy with the return code). Here, the information goes in the other direction and the constraints are different. This leads to a much simpler protocol. There is also a part for the preparation of the finalization code, which is even simpler.

Before explaining the details of how all of this is done, we present the cryptographic building blocks that are required.

## 3.2 Cryptographic building blocks

Let $\mathbb{G}$ be a cyclic group with generator $g$ of prime order $q$, in which the Decisional Diffie-Hellman problem is assumed to be difficult. The candidates are encoded by a set of $n$ distinct and independent generators $v_1,\dots,v_n$ of the group $\mathbb{G}$. The correspondence between the candidates and the $v_i$'s is public; it can be the result of a hash function that maps to $\mathbb{G}$. From now on, we often use only the $v_i$'s, keeping this correspondence implicit.

### 3.2.1 ElGamal encryption

Our protocol uses ElGamal encryptions. We denote by $\{M\}_{\mathsf{pk}}$ a ciphertext $(g^r, M\,\mathsf{pk}^r)$, where $r$ is a fresh random element of $\mathbb{Z}_q$, $M\in\mathbb{G}$ is the plaintext, and $\mathsf{pk}$ is the public key. The discrete logarithm $\mathsf{sk}$ of $\mathsf{pk}$ is the secret key that allows to decrypt. As a public-key encryption scheme, ElGamal is Ind-CPA secure under the DDH assumption. It supports a basic homomorphic property which allows in particular to easily re-randomize a ciphertext (without knowing $\mathsf{sk}$), by point-wise multiplication with an encryption of the neutral in $\mathbb{G}$. More generally, the homomorphic property leads to simple and efficient zero-knowledge proofs (ZKP) for algebraic properties related to ciphertexts. We will list the ones we use below.

If several parties each have a public key $\mathsf{pk}_j$, then, one can form the agglomerated public key $\mathsf{pk}=\prod\mathsf{pk}_j$. The corresponding secret key $\mathsf{sk}$ is the sum of the secret keys $\mathsf{sk}_j$ of all parties. A ciphertext $(c_1,c_2)$ encrypted with $\mathsf{pk}$ can be decrypted in a distributed manner, where each party computes and sends $c_1^{\mathsf{sk}_j}$. Multiplying all these contributions gives $c_1^{\mathsf{sk}}$, and $c_2/c_1^{\mathsf{sk}}$ is the plaintext. Adding a threshold feature (for robustness) to this simple distributed protocol is well known; see [3] for a modern treatment in the context of e-voting. For simplicity, we stick to the basic $n$-out-of-$n$ version.

### 3.2.2 Classical zero-knowledge proofs

We list here some classical ZKPs that are used in our protocol, without recalling how they work, since they can be found in many places [1, 22, 32]. These are based on Sigma protocols turned into interactive versions via the Fiat-Shamir transform. All of them can be seen as instances of the general framework by Maurer [27], that we briefly recall in Appendix C.1. During the transform, it is important to hash as much context as possible: the election identifier, the complete public data required to verify the ZKP, the identity of the prover, and the precise step of the protocol where the proof is performed. This prevents any replay of the ZKP by a malicious party.

- $\mathsf{ZKP}_{\mathsf{DL}}$: Given $K=g^k$, this proves knowledge of $k$, the discrete logarithm of the public element $K$ (e.g. a public key).
- $\mathsf{ZKP}_{\mathsf{rand}}$: Given a ciphertext $\{M\}_{\mathsf{pk}}=(g^r, M\,\mathsf{pk}^r)$ and the public key $\mathsf{pk}$, this proves knowledge of $r$, the randomness used during the encryption. This implies the knowledge of the plaintext $M$.
- $\mathsf{ZKP}_{\mathsf{encDL}}$: Given an encryption of a power $\{g^M\}_{\mathsf{pk}}=(g^r, g^M\,\mathsf{pk}^r)$ and the public key $\mathsf{pk}$, this proves knowledge of the randomness $r$, and of the discrete logarithm $M$ of the plaintext.
- $\mathsf{ZKP}_{\mathsf{exp}}$: Given $K=g^k$, $h$ and $H=h^k$, this proves that $H$ has indeed be obtained by exponentiating the group

element $h$ by $k$, the discrete logarithm of the public element $K$ (e.g. during a proof of correct decryption).

In addition, we will also need the following, which is less standard. We give an algorithm for it in Appendix C.2. We remark that this is a zero-knowledge argument (and not a proof), because the zero-knowledge property relies on DDH.

- $\mathsf{ZKP}_{\mathsf{ptxte}}$: Given $c_1 = \{v\}_{\mathsf{pk}}$, $c_2 = v^k$ and $c_3 = g^k$, this proves that $c_2$ is equal to the plaintext $v$ corresponding to the ciphertext $c_1$ raised to the power $k$, where $k$ is the discrete logarithm of $c_3$.

#### 3.2.3 Plaintext equivalence test (PET)

Let $\mathsf{pk}$ be a public key, the secret key of which is shared as $\mathsf{sk} = \sum \mathsf{sk}_j$ between $m$ parties, and let $c_1 = \{M_1\}_{\mathsf{pk}}$ and $c_2 = \{M_2\}_{\mathsf{pk}}$ be two ciphertexts. A PET is a protocol run between the parties to jointly decide (and prove to a third-party) whether $M_1 = M_2$, without anyone learning anything about $M_1$ and $M_2$ except this bit of information, including the holders of the secret key, as soon as one of them is honest.

The general idea is to mask the plaintexts using the homomorphic property of ElGamal and to combine them into a ciphertext that, after decryption, gives either the neutral element, or a uniform element of the group, depending on the $M_1 = M_2$ bit. The details can be found for instance in [28].

#### 3.2.4 Verifiable mixnets and variants

Let $[c_i]_{1\le i\le n}$ be a list $L$ of $n$ ciphertexts for a public key $\mathsf{pk}$. A shuffle operation consists in re-encrypting each $c_i$ into another ciphertext $\tilde{c}_i$, and applying a random permutation $\sigma$ to the list, leading to $\tilde{L} = \left[\tilde{c}_{\sigma(i)}\right]_{1\le i\le n}$. When $L$, $\tilde{L}$ and $\mathsf{pk}$ are public, it is possible for the entity who performed the shuffle to publish a ZKP that proves that the multiset of cleartexts corresponding to $L$ and $L'$ are the same. The algorithm of Terelius and Wikström [33] is classical and widely used in e-voting.

When several servers do such shuffling sequentially to a list of ballots to be tallied, this lead to a so-called verifiable mixnet, so that the final output list contains the same votes, but the link with the Voters is cut, as soon as one of the servers is honest.

In our protocol, we use verifiable mixnets also during the setup phase, where the list of candidates is shuffled. In fact, we need a primitive that does slightly more.

Let $\mathsf{pk}_1$ and $\mathsf{pk}_2$ be two public keys, and $K = g^k$ a public group element whose discrete logarithm is known by the shuffler. Let $L = \left[\{x_i\}_{\mathsf{pk}_1}, \{y_i\}_{\mathsf{pk}_2}\right]_{1\le i\le n}$ be a list of pairs of ciphertexts for both keys. The shuffler re-encrypts the two ciphertexts, and raise the second one to the power $k$ (thus raising the associated plaintext to the same power, due to the homomorphic property). Then it applies a random permutation $\sigma$ to the new pairs, and forms the output list $\tilde{L} = \left[\{\tilde{x}_{\sigma(i)}\}_{\mathsf{pk}_1}, \{\tilde{y}^k_{\sigma(i)}\}_{\mathsf{pk}_2}\right]_{1\le i\le n}$. A ZKP is added, that proves that this has been performed as prescribed, without revealing the intermediate steps. This is not standard, but can be obtained with a cheap modification of the Terelius-Wikström approach. The details are given in Appendix C.3.

We will call this procedure DbleMixExp.

### 3.3 Setup phase

**Parties** : SC (party 0) and the $\mathsf{CC}_j$ (parties $1, 2, \ldots, m$)
**Context** : $\mathsf{pk}_{\mathsf{CC}}$, $\mathsf{pk}_{\mathsf{PS}}$, $[v_i]_{1\le i\le n}$
**Output** : $\left[\{v_{\sigma(i)}\}_{\mathsf{pk}_{\mathsf{PS}}}, \{v^{\mathsf{sv_{id}}}_{\sigma(i)}\}_{\mathsf{pk}_{\mathsf{CC}}}\right]_{1\le i\le n}$, $\mathsf{pv_{id}}$, $\pi$
*(where $\sigma$ and $\mathsf{sv_{id}}$ are known by no party as soon as one of them is honest. The data $\pi$ is auxiliary data for verifiability.)*

1 **Initialization:**
2 Parties compute $L_{-1} = \left[\{v_i\}_{\mathsf{pk}_{\mathsf{PS}}}, \{v_i\}_{\mathsf{pk}_{\mathsf{CC}}}\right]_{1\le i\le n}$, where the encryptions are with randomness 0.
3 They set $\mathsf{ppv}_{\mathsf{id},-1} = g$.
4 **Round of computation-communication:**
5 For $j$ from 0 to $m$, party $j$ does the following:
6 Picks $\sigma_j$, a random permutation, and $\mathsf{sv_j}$, a random exponent in $\mathbb{Z}_q$.
7 Computes $\mathsf{pv_j} = g^{\mathsf{sv_j}}$.
8 Computes $\mathsf{ppv}_{\mathsf{id,j}} = {\mathsf{ppv}_{\mathsf{id,j-1}}}^{\mathsf{sv_j}}$, with $\mathsf{ZKP}_{\mathsf{exp}}$.
9 Applies DbleMixExp to $L_{j-1}$, with parameters $\sigma_j$ and $\mathsf{sv_j}$, to produce $L_j$, with associate ZKP $\pi_j$.
10 The data $(L_j, \pi_j, \mathsf{pv_j}, \mathsf{ppv}_{\mathsf{id,j}})$ is broadcast.
11 **Finalization:**
12 All the parties check all the ZKPs that they did not yet check.
13 The final list is $L_m$, which has the expected format, with $\sigma = \sigma_m \circ \cdots \circ \sigma_1 \circ \sigma_0$ and $\mathsf{sv_{id}} = \mathsf{sv}_0 \cdot \mathsf{sv}_1 \cdots \mathsf{sv_m}$; the final $\mathsf{ppv}_{\mathsf{id,m}}$ is $\mathsf{pv_{id}}$.
14 The output $\pi$ is formed of all the data that has been broadcast.

**Algorithm 1:** Masking choices sub-protocol

The setup phase involves the Setup Component SC, the $m$ Control Components $[\mathsf{CC}_j]_{1\le j\le m}$, and the Printer. Before running the setup phase, everyone should know the public keys of all actors. During this phase, all messages are broadcast, and must be signed by the emitter. We don't make the signatures explicit. However, encryptions are explicit, and we denote by $\mathsf{pk}_{\mathsf{PS}}$ the public key of the Printer, and $\mathsf{pk}_{\mathsf{CC}}$ a public key whose associated secret key is shared between the CCs.

The set of all the communications is seen as a transcript that is recorded and checked later on by auditors, before the voting phase starts.

As sketched in Subsection 3.1, the basis of the setup phase

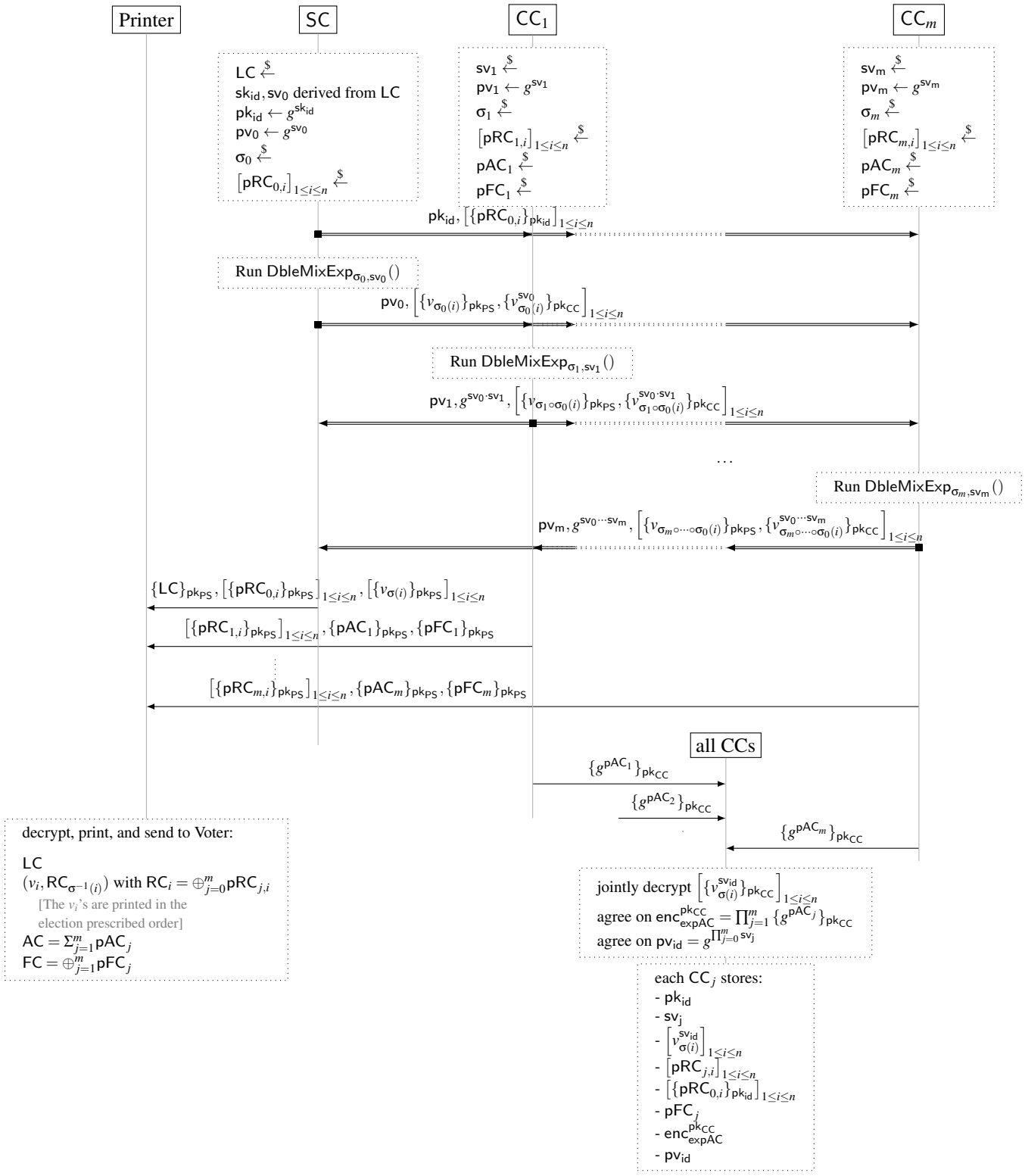


Figure 3: Setup phase. This shows the operation for one Voter. The real protocol runs this in parallel for all Voters.
All zero-knowledge proofs and corresponding checks are omitted. $\sigma$ is defined as $\sigma_m \circ \cdots \circ \sigma_0$. The doubled arrows are broadcast communications.

**Parties** : SC (party 0) and the $\mathsf{CC}_j$ (parties $1, 2, \ldots, m$)
**Context** : $\mathsf{pk}_{\mathsf{CC}}$, $\mathsf{pk}_{\mathsf{PS}}$, $[v_i]_{1\le i\le n}$
**Output** : data for the Printer: $\{\mathsf{LC}\}_{\mathsf{pk}_{\mathsf{PS}}}$, $\left[\{v_{\sigma(i)}\}_{\mathsf{pk}_{\mathsf{PS}}}\right]_{1\le i\le n}$, $\left[\{\mathsf{pRC}_{0,i}\}_{\mathsf{pk}_{\mathsf{PS}}}\right]_{1\le i\le n}$, …, $\left[\{\mathsf{pRC}_{m,i}\}_{\mathsf{pk}_{\mathsf{PS}}}\right]_{1\le i\le n}$.
data for $\mathsf{CC}_j$: $\mathsf{pk}_{\mathsf{id}}$, $\mathsf{pv}_{\mathsf{id}}$, $\mathsf{sv}_{\mathsf{j}}$, $\left[v_{\sigma(i)}^{\mathsf{sv}_{\mathsf{id}}}\right]_{1\le i\le n}$, $\left[\mathsf{pRC}_{j,i}\right]_{1\le i\le n}$, $\left[\{\mathsf{pRC}_{0,i}\}_{\mathsf{pk}_{\mathsf{id}}}\right]_{1\le i\le n}$.
data for auditors.

**Generation of login data:**
SC picks LC, a 128-bit long nonce and derives from it a secret key $\mathsf{sk}_{\mathsf{id}}$.
SC broadcasts $\mathsf{pk}_{\mathsf{id}} = g^{\mathsf{sk}_{\mathsf{id}}}$, with a $\mathsf{ZKP}_{\mathsf{DL}}$.
SC encrypts LC for the Printer, with a $\mathsf{ZKP}_{\mathsf{rand}}$.
**Generation of return code data:**
Run the masking choices sub-protocol, where SC, instead of picking a random $\mathsf{sv}_0$, derives it from LC.
The $\mathsf{CC}_j$ for $1 \le j \le m$ jointly decrypt $\{v_{\sigma(i)}^{\mathsf{sv}_{\mathsf{id}}}\}_{\mathsf{pk}_{\mathsf{CC}}}$.
For $0 \le j \le m$, party $j$ (SC or $\mathsf{CC}_j$) picks $\left[\mathsf{pRC}_{j,i}\right]_{1\le i\le n}$, a list of short random partial return codes, and encrypts them with $\mathsf{pk}_{\mathsf{PS}}$, with a $\mathsf{ZKP}_{\mathsf{rand}}$.
SC also encrypts $\left[\mathsf{pRC}_{0,i}\right]_{1\le i\le n}$ with $\mathsf{pk}_{\mathsf{id}}$, together with a $\mathsf{ZKP}_{\mathsf{rand}}$.

**Algorithm 2:** Return code setup sub-protocol

is a sub-protocol for masking the choices $v_1$, …, $v_n$. This is described as Algorithm 1, whose main part is a round of DbleMixExp between all the parties. A complication when turning this into a full return code sub-protocol is that the SC is offline during the voting phase. The strategy to solve this is to use a so-called login code, printed on the voting card, that contains enough entropy to serve as a secret key, thus creating a secure channel to the Voter. This becomes Algorithm 2.

In addition to the return code setup sub-protocol, the setup phase includes sub-protocols to generate data for the approve code and the finalization code.

The finalization code is handled similarly as the return codes, but it is much simpler because there is no associated vote that needs to be hidden, so that the masking part disappears. Also, SC does not need to be involved in the generation of this code.

As for the approve code, this goes in the other direction: this is a short code (typically 9 digits), denoted by AC, that the Voter reads on their voting card and types after having checked that the return code is correct. It is generated during the setup phase as the sum of partial approve codes $\mathsf{pAC}_j$ picked at random by $\mathsf{CC}_j$ and sent (encrypted) to the Printer.

Furthermore, a small protocol is run between the $\mathsf{CC}_j$ so

**Parties** : The $\mathsf{CC}_j$ (parties $1, 2, \ldots, m$)
**Context** : $\mathsf{pk}_{\mathsf{CC}}$, $\mathsf{pk}_{\mathsf{PS}}$
**Output** : $\left[\{\mathsf{pAC}_j\}_{\mathsf{pk}_{\mathsf{PS}}}, \{\mathsf{pFC}_j\}_{\mathsf{pk}_{\mathsf{PS}}}\right]_{1\le j\le m}$, $\mathsf{enc}_{\mathsf{expAC}}^{\mathsf{pk}_{\mathsf{CC}}} = \{g^{\Sigma \mathsf{pAC}_j}\}_{\mathsf{pk}_{\mathsf{CC}}}$, data for auditors.

For $1 \le j \le m$:
  $\mathsf{CC}_j$ picks $\mathsf{pAC}_j$ and $\mathsf{pFC}_j$ at random.
  $\mathsf{CC}_j$ broadcasts $\{\mathsf{pAC}_j\}_{\mathsf{pk}_{\mathsf{PS}}}$ with a $\mathsf{ZKP}_{\mathsf{rand}}$.
  $\mathsf{CC}_j$ broadcasts $\{g^{\mathsf{pAC}_j}\}_{\mathsf{pk}_{\mathsf{CC}}}$ with a $\mathsf{ZKP}_{\mathsf{encDL}}$.
  $\mathsf{CC}_j$ broadcasts $\{\mathsf{pFC}_j\}_{\mathsf{pk}_{\mathsf{PS}}}$ with a $\mathsf{ZKP}_{\mathsf{rand}}$.
Each $\mathsf{CC}_j$ computes $\mathsf{enc}_{\mathsf{expAC}}^{\mathsf{pk}_{\mathsf{CC}}} = \prod_{j=1}^{m} \{g^{\mathsf{pAC}_j}\}_{\mathsf{pk}_{\mathsf{CC}}}$ and broadcasts a signature of it.
Each $\mathsf{CC}_j$ checks the ZKPs and signatures of others.

**Algorithm 3:** Approve and finalization codes setup sub-protocol

that they all get $\{g^{\mathsf{AC}}\}_{\mathsf{pk}_{\mathsf{CC}}}$.

The details are given in Algorithm 3.

We summarize all the parts of the setup phase in Figure 3, where we slightly interleave the sub-protocols, and we skip the zero-knowledge proofs for readability. This is run simultaneously for all the Voters. The sequence of computations / communications is as follows:

- SC initiates the protocol and sends data to the CCs and the Printer. Thereafter, it does nothing, except checking the validity of the transcript (which can be done also by an auditor).
- Then, the CCs make a round of computations / communications for the masking choices sub-protocol, and they do the rest with a few independent broadcasts between them. They finally send data to the Printer.
- At the end, the Printer decrypts the data that it received, with which it prepares the voting material: for each Voter, it computes $\mathsf{RC}_i = \oplus_{j=0}^{m} \mathsf{pRC}_{j,i}$, $\mathsf{AC} = \Sigma_{j=1}^{m} \mathsf{pAC}_j$, and $\mathsf{FC} = \oplus_{j=1}^{m} \mathsf{pFC}_j$. After associating $\mathsf{RC}_i$ to $v_{\sigma(i)}$, it organizes the layout as in Figure 1, prints the voting cards, and sends them by postal mail.

## 3.4 Voting phase

### 3.4.1 Return code sub-protocol

The Voter starts by typing their login code LC on their Client. From it, the Client derives $\mathsf{sk}_{\mathsf{id}}$ and $\mathsf{sv}_0$. The Voter also chooses a candidate $v$ among the $n$ possibilities.

Then, a secure channel (authenticated on both sides and secret) is established between the Client and each CC. This is based on the Client knowing the public key of each CC, and the CCs knowing $\mathsf{pk}_{\mathsf{id}}$, a public key corresponding to $\mathsf{sk}_{\mathsf{id}}$.

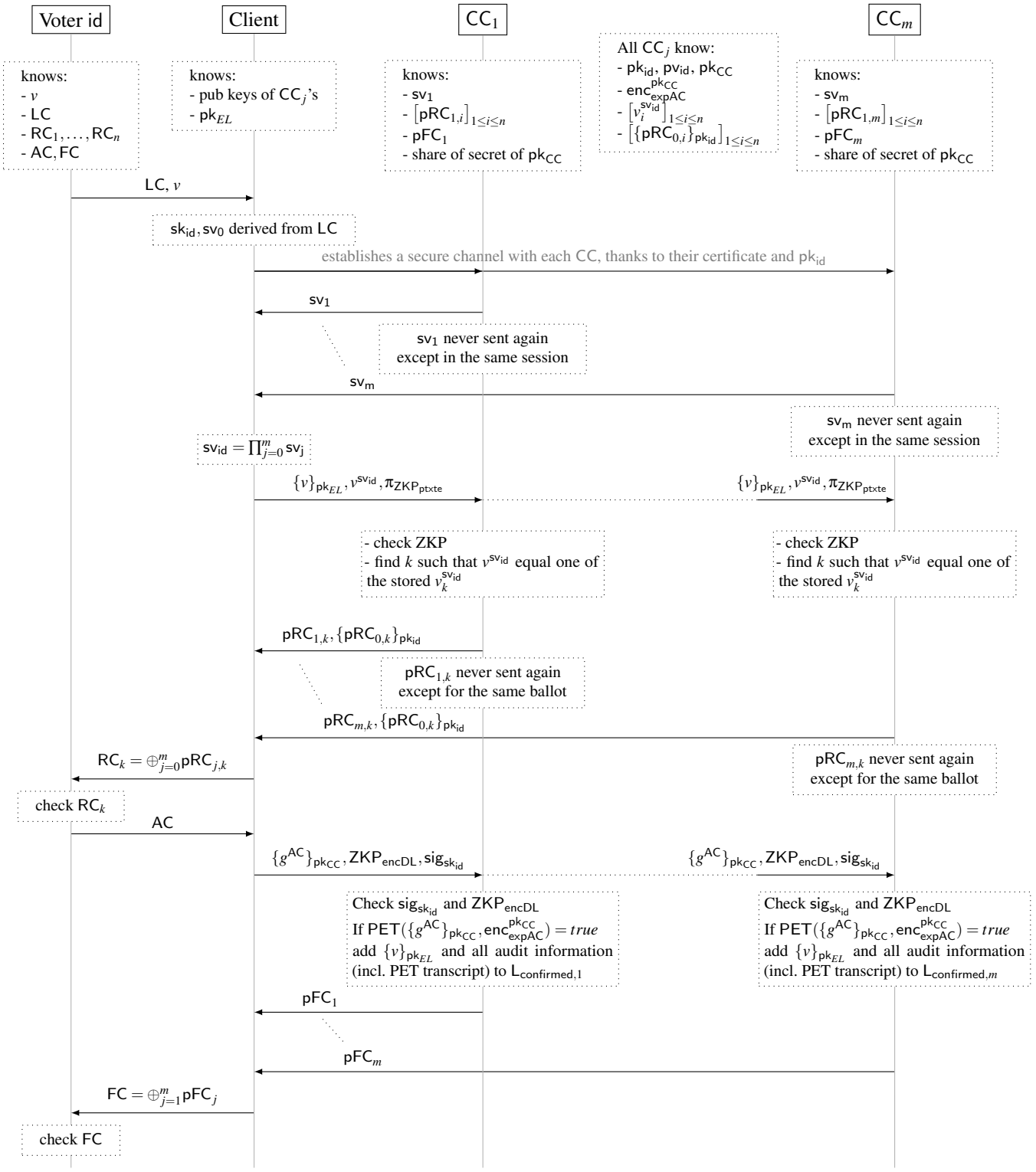


Figure 4: Voting phase.

This allows the CCs to send the $\mathsf{sv}_j$'s for $j = 1, \ldots, m$ to the Client, which can then deduce $\mathsf{sv}_{\mathsf{id}} = \prod_{0 \le j \le m} \mathsf{sv}_j$. The secure channels are no longer needed thereafter.

The Client encrypts the vote $v$ with $\mathsf{pk}_{EL}$, and computes a masked version of it $v^{\mathsf{sv}_{\mathsf{id}}}$, together with a ZKP that the same $v$ is used in both parts. This is sent to the CCs who check the ZKP.

Each $\mathsf{CC}_j$ looks in its table $\left[v^{\mathsf{sv}_{\mathsf{id}}}_{\sigma(i)}\right]_{1 \le i \le n}$ which index $k$ contains the $v^{\mathsf{sv}_{\mathsf{id}}}$ send by the Client. It sends back the corresponding partial return code $\mathsf{pRC}_{j,k}$. It also sends $\mathsf{pRC}_{0,k}$, encrypted with $\mathsf{pk}_{\mathsf{id}}$, that they got from the SC during the Setup.

With this information, the Client reconstructs the return code $\mathsf{RC}_k$ and shows it to the Voter, who can check that this is the one written on the voting card for the candidate $v$. If this is not the case, the protocol is aborted, and the Voter has to complain.

#### 3.4.2 Approval sub-protocol

When happy with the return code, the Voter reads the approve code AC on the voting card, and types it on the Client. The Client computes $g^{\mathsf{AC}}$, encrypts it with $\mathsf{pk}_{\mathsf{CC}}$, produces a proof of knowledge of the discrete logarithm of the plaintext, and signs this with $\mathsf{sk}_{\mathsf{id}}$.

The CCs, as a whole, could decrypt this and decrypt the version they had kept from the setup phase. However, we do not want them to learn the value of $g^{\mathsf{AC}}$ for the following reason: in the case where the Voter mistype AC, they should be allowed a second try, and we don't want that after the first trial a dishonest CC could impersonate the Voter and confirms the vote in their place. For this reason, we resort to a PET protocol in order to compare the plaintexts without decrypting.

If the PET confirms that the AC sent by the Voter is correct, then $\{v\}_{\mathsf{pk}_{EL}}$ is added to the list of ballots to be tallied, and audit information is kept, containing the transcript of all the communications.

Then the partial finalization codes are sent to the Client that reconstructs the finalization code which should be the same as the one the Voter can read on the voting card.

The whole voting phase protocol is summarized in Figure 4.

### 3.5 Tally

The tally phase is very classical: the Talliers proceed with a round of verifiable mixnets on the list of $\{v\}_{\mathsf{pk}_{EL}}$'s to be tallied, and then performs a verifiable decryption of them using their shared secret key.

A small complication arises, due to the absence of a public bulletin board. At the end of the voting phase, each $\mathsf{CC}_j$ has a list $\mathsf{L}_{\mathsf{confirmed},j}$ of ballots that must be tallied, together with audit data. They send this information to the Talliers and to the Auditors who check the validity of the audit data.

In the case of disagreement between the lists, it is safe to take the union of them. Indeed, if a ballot is present in the list of one of the CCs, then the associated audit data contains the transcript of the PET protocol, that ensures that all the CCs participated to the voting phase for this ballot, and that they should all have included it in their final list.

We remark that such a pre-tally consensus procedure, which is required to avoid some verifiability attacks, has been added only recently in the Swiss Post protocol and is still missing in CHVote.

### 3.6 Complexity

The computational cost is dominated by group exponentiations, whose number grows, overall, linearly with the number $m$ of CCs, the number $n$ of candidates, and the number $N$ of Voters. We denote by $n_{\text{bal}}$ the number of cast ballots, which is smaller than $N$ if there is a low turnout. In practice, $m$ remains small, but $n$ can be somewhat large. We therefore focus on the dominant term of the form $(k_1 m + k_2)nN$, and make the constants $k_1$ and $k_2$ explicit for the different phases and parties. During the voting phase, the costs are small, and we consider them negligible for the CCs; however, we make them explicit for the Client, which has limited computing power.

The count is given in Appendix D and is summarized in Figure 5.

| | SC / Tallier | $\mathsf{CC}_j$ / Tallier | Client |
|---|---|---|---|
| Setup | $(15m+23)nN$ | $(15m+22)nN$ | — |
| Voting | — | negl. | $m$ sec. chan.<br>22 |
| Tally | $(12m+13)n_{\text{bal}}$ | | — |

Figure 5: Complexities for each party in number of group exponentiations. We assume here that there are $m+1$ Talliers, typically SC and all the CCs.

#### 3.6.1 Practicality

The setup phase is the most costly, and we study its practical feasibility.

In the Swiss context, $m = 4$, and $n$ can sometimes be large. For $n = 100$, we get a cost of $8,300$ exponentiations per Voter. If there are $N = 100,000$ Voters, this requires $8.3 \cdot 10^8$ exponentiations. Assuming a computing core can perform $5,000$ exponentiations per second (this is the case with optimised libraries using elliptic curves), we get 46.1 core-hours. This can be easily parallelised, and if all the components are 10-core servers, the setup phase can be run in less than 5 hours. With the same data and a turnout of 50%, we get 10.2 core-minutes for each Tallier during the tally phase.

Another way to assess practicality is to compare the costs with those of the Swiss Post protocol, which is deployed in several cantons (and is therefore practical). The higher trust in SC makes things simpler, and therefore cheaper. Still, the cost of the setup phase for SC is of $(2m+4)nN$ exponentiations. Our protocol therefore requires roughly 7 times more exponentiations than theirs. On the other hand, their protocol relies heavily on the structure of the multiplicative group of finite fields, leading to exponentiations that are much more expensive than in our case, where we can use elliptic curves. We estimate that this should compensate, and that the cost for SC is in the same ballpark for both protocols. For the CCs, the comparison is unfair, because in the Swiss Post protocol, they do not need to check the ZKPs of the others (they trust SC for doing it). But, if we assume that in our protocol the CCs delegate these verifications to Auditors, then, again the costs are similar.

#### 3.6.2 Case of multiple questions / multiple answers

In the above estimates, we have assumed that there is a single question, and a single answer, to be chosen among $n$ candidates. When there are multiple questions, each of them allowing multiple answers, the complexities are modified as follows.

During the setup phase, $n$ should be replaced by $n_{\text{tot}}$, the sum of the number of candidates for all questions; the cost remains linear in the number of items printed and sent to Voters.

During the voting phase, the cost of the first communication of the Client is multiplied by $n_{\text{ans}}$, the sum of the number of answers for all questions. This remains linear in the number of actions made by the (human) Voter (selecting a candidate and checking the corresponding return code).

For the tally phase, the Talliers need to deal with a list of tuples of ciphertexts. Therefore the cost is roughly multiplied by $n_{\text{ans}}$. This is actually slightly less, because the costs related to the permutation in the ZKP of the shuffles are shared. We acknowledge that for this phase our solution is slower than [32] and [22], where multiple answers are encoded in a single ciphertext. Their approach seems difficult to adapt to a generic group. We don't expect this to be a practical issue, because this remains smaller than the cost of the setup phase.

## 4 Security proofs

We analyze our protocol using the tool ProVerif [6].

### 4.1 Brief overview of ProVerif

ProVerif analyzes protocols in a symbolic model, where messages are represented by terms, that intuitively reflect the structure of the messages but abstract away most of the underlying algebraic properties. In particular, the ElGamal encryption is represented by a symbol `aenc` and the attacker may only retrieve the plaintext when it has the decryption key. We do not consider the homomorphic property of ElGamal. While this may miss some attacks, such an abstraction is usual in symbolic models and allows for a high level of automation in the security proofs.

Protocol roles are modeled as processes, that are a list of commands executed by the participants. For example, `out(c,m)` sends the message `m` over the channel `c`, while `in(c,x)` inputs a message on channel `c` and stores it in variable `x`. Processes are typically executed in parallel, with an arbitrary interleaving left under the control of the attacker. The attacker may intercept any message sent over a public channel and send any message that it can build. To model that some component is corrupted (for example the Client or a CC), it suffices to output all the secret values of the component to the attacker.

We refer the reader to [6] for a more complete description of ProVerif and its semantics. Other tools such as Tamarin [29] or CPSA [26] are also used to formally analyze protocols in symbolic models. We chose ProVerif since it has already been used for analyzing privacy and verifiability of several voting protocols (e.g. [9, 16]) and in particular in the Swiss context [4, 14, 31].

Recently, a framework has been designed in ProVerif to analyze evoting protocols and prove they ensure End-To-End verifiability [12]. We decided not to use this framework to keep the verifiability properties studied closer to the ones mentioned by the Swiss Federal regulation, and keep the verifiability and privacy models aligned (at the time of writing the framework does not support vote privacy).

### 4.2 Protocol model

Each role of our protocol needs to be modeled as a ProVerif process. We provide an excerpt of the Client below. The Client first gets LC and the voting choice $v$ from the Voter. The communication between the Voter `voter_name` and the Client is modeled as a channel `c_voter_to_VD(voter_name)`, private between the Voter and the Client since this communication cannot be observed by an attacker. The Client derives $\mathsf{sk}_{\mathsf{id}}$ and $\mathsf{sv}_0$ from LC. It retrieves the other $\mathsf{sv}_{\mathsf{id}\,j}$'s from each online CC and then builds the ballot as expected.

```
let VotingDevice(voter_name) =
    let pkEL = get_pkEL() in
    in(c_voter_to_VD(voter_name),(LC,v));
    let sv_0 = kdf_sv(LC) in
    let sk_id = kdf_sk_id(LC) in
    in(c_voter_to_CCj(voter_name,CC1),sv_1);
    in(c_voter_to_CCj(voter_name,CC2),sv_2);
    let prod_sv = Prod(sv_0,sv_1,sv_2) in
    let pv = exp(G,prod_sv) in

    (* Start CreateVote *)
    new r;
    let c1 = aenc(pkEL,v,r) in
    let c2 = exp(v,prod_sv) in
    let pi = zkp_ptxte(
        v,prod_sv,r,
```

```
            c1,c2,pv,pkEL
            ) in
        let ballot = (c1,c2,pi) in
        out(cpub,ballot);
        (* End CreateVote *)
```

Functions such as `aenc`, `exp`, and `zkp_ptxte` are just function symbols, whose properties are defined through equations. For example, a decryption function `adec` is associated with the asymmetric encryption function `aenc`, with the corresponding equation that models decryption:

```
adec(aenc(pkey(sk),x,r), sk) = x
```

The equation associated to the zero-knowledge proof `zkp_ptxte(...,c1,c2,...)` says that the proof is valid only if `c2` is the exponentiation of the plaintext contained in `c1`.

```
let pv = exp(G,Prod(k1,k2,k3)) in
let pi = zkp_ptxte(
    x,Prod(k1,k2,k3),r,
    aenc(pk,x,r),exp(x,Prod(pk,k2,k3)),pv,pk
  ) in
verify_zkp_ptxte(
  pi, aenc(pk,x,r),
  exp(x,Prod(k1,k2,k3)),pv,pk
) = true
```

As said earlier, we only provide a minimal set of equations for `exp`, in order to reflect that when the CCs successively exponentiate `aenc(pk,v,r)` with the $\mathsf{sv}_j$, it yields, after decryption, `exp(v,Prod(sv_0,sv_1,sv_2))`.

## 4.3 Security properties

We analyze vote privacy and verifiability. For vote privacy, we consider the well established property [16] that defines privacy as an equivalence property: an attacker should not be able to distinguish the scenario where Alice votes 0 and Bob votes 1, from the converse one where Alice votes 1 and Bob votes 0.

$$\mathsf{Voter}(\mathsf{Alice},0) \mid \mathsf{Voter}(\mathsf{Bob},1) \approx \mathsf{Voter}(\mathsf{Alice},1) \mid \mathsf{Voter}(\mathsf{Bob},0)$$

This is expressed in ProVerif through diff-equivalence [7]

```
Voter(Alice,diff[ZERO,ONE]) | Voter(Bob,diff[ONE,ZERO])
```

When ProVerif proves such a diff-equivalence, then the process obtained by considering the left parts of the `diff` is equivalent to the process obtained by considering the right parts of the `diff`.

To express verifiability, we annotate the process with events that do not change the behavior. For example, we add `event HappyVoter(id,vo)` at the end of the process `Voter`, when the Voter has successfully performed all their checks. Then Recorded-as-intended ensures that whenever a Voter voting for $v$ successfully finishes the voting procedure, then the system should have registered a ballot $c$ that contains $v$. This is expressed by the following property.

```
event(HappyVoter(id,v))
   ==> event(BallotRegistered(c)) && c = aenc(pk,v,r).
```

Similarly, eligibility ensures that whenever a ballot $c$ is about to be counted (`Going_to_tally(id,c)`), then it comes from a legitimate Voter. We can actually show a stronger property, that we call *strong eligibility*, that further ensures that any ballot that comes from an honest Voter does contain their vote. This guarantees that the attacker cannot modify the vote. Strong eligibility is modeled as follows.

```
event(Going_to_tally(id,c)) ==>
  event(Corrupted(id))
  || (event(Honest(id))
      && event(PreConfirmed(id,v))
      && c = aenc(pk,v,r)).
```

The event `PreConfirmed(id,v)` corresponds to the stage where the Voter has confirmed the return codes by sending their approval code AC. We cannot guarantee that the Voter has successfully received the finalization code FC since this last communication may always be stopped by an attacker.

Universal verifiability ensures that the result of the election corresponds to the contents of the registered ballots. It directly follows from the soundness of the zero-knowledge proofs of mixing and decryption. Since these primitives are sound by construction in our symbolic models, we do not prove universal verifiability in ProVerif and we rely instead on computational proofs [33].

## 4.4 Modeling choices

To simplify the analysis, we consider only two Control Components, $\mathsf{CC}_1$ and $\mathsf{CC}_2$, to cover two extreme cases: either the first CC is compromised or the last one.

*Arbitrary number of candidates.* One first main issue when modeling our protocol is the fact that the number of candidates is an election parameter. To avoid limiting the analysis to 2 or 3 candidates, we model elections with an arbitrary number of candidates, chosen by the adversary during the setup. The difficulty is that Voters receive a voting sheet with the list of candidates and need to check the right voting code, while this sheet is of arbitrary size. To circumvent this issue, we consider a Printer that (virtually) generates the voting sheet in several pieces, one piece for each voting option, by adding each piece to a table

```
paper_sheet(voter_name,LC,voting_option,RC,AC,FC);
```

We then ensure consistency between the pieces with restrictions [8] in order to make sure that the authorities use the same data for a given Voter.

*Mixnets.* Then the second and probably most important issue was due to mixnets. Most voting protocols include a mixnet but only during the tally phase. This mixing step is typically modeled as follows:

```
let Tally() =
  get ballot_box(=Alice,cA) in
  get ballot_box(=Bob,cB) in
  let resA = adec(cA,skEL) in
  let resB = adec(cB,skEL) in
  out(cpub,resA) |  out(cpub,resB)
```

| | VD | SC | CCj | Old FCh req. | New FCh req. | New protocol |
|---|---|---|---|---|---|---|
| recorded-as-intended | D | H | $\geq 1$ | required | required | ✔ |
| | D | D | $\geq 1$ | – | required | ✔ |
| | D | H | D | – | – | ✘ |
| eligibility | H | D | $\geq 1$ | – | required | ✔$^+$ |
| | H | H | D | – | – | ✔$^+$ |
| | D | D | $\geq 1$ | – | required | ✔$^+$ |
| | D | H | D | – | – | ✔ |
| vote privacy | H | D | $\geq 1$ | – | required | ✔ |
| | H | H | D | required | required | ✔ |

Table 1: Results of the analysis in ProVerif and comparison with old and new versions of the FCh requirements.
–: not required ✔: proved ✔$^+$: strong eligibility (provably) holds

The parallel operator | models the mixing: the two results `resA` and `resB` are output in any order. In particular, we have

$$\mathsf{out}(c,0) \mid \mathsf{out}(c,1) \approx \mathsf{out}(c,1) \mid \mathsf{out}(c,0)$$

This issue here lies in the fact that ProVerif proves a stronger notion of equivalence, based on the structure of the process, that tries to relate strictly each sub-component of the process. In particular, ProVerif will try to prove $\mathsf{out}(c,0) \approx \mathsf{out}(c,1)$, which does not hold. This issue can be easily solved as usual, by replacing the last line of the `Tally` process by

```
out(cpub,diff[resA,resB]) | out(cpub,diff[resB,resA])}
```

which tells ProVerif that `resA` and `resB` can safely be swapped.

However, in our protocol, we not only have mixnets during the tally phase but also during the setup phase. Indeed, the return codes are mixed so that the CCs do not learn the vote when they compute the code during the voting phase. This mix is encoded using a table `mixnet`: the authority adds all the ciphertexts that should be mixed to the table and then reads (non deterministically) the ciphertexts, which ensures mixing. However, to be able to *prove* equivalence, we need to guide ProVerif. More specifically, we need to tell ProVerif that for any permutation of the return codes for Alice that for example places the return code for candidate 0 in 2nd position, then there is another possible permutation of the return codes that places the return code for candidate 1 in 2nd position. This is done by introducing a function `Shuffle_and_exp_CC0` (and similarly for other CCs) that defines a correspondence between the mixing permutations.

### 4.5 Results

All our models are available as artifacts [2]. We wrote our privacy and verifiability models in order to minimize the difference between the two, ensuring that the protocol is analyzed under the same modeling assumptions.

The security analysis has been conducted on a Macbook Pro - M2 Pro, 32Go RAM, and requires up to 5h and 20GB.

We obtain that our protocol is secure under the trust assumptions considered by the (novel) Swiss requirements. Namely, our protocol ensures privacy as soon as either SC or one of the CCs is honest, thanks to the fact that the honest authority has permuted the return codes and detains a private share of $\mathsf{sv}_{\mathsf{id}}$. Similarly, strong eligibility holds as soon as either SC or one of the CCs is honest. Our protocol actually goes beyond the Swiss requirements. When all the CCs are dishonest, it still ensures eligiblity: the CCs cannot add valid ballots on behalf of absentee Voters and for Voters with honest Clients, they cannot modify their vote. Finally, recorded-as-intended holds as soon as one of the CCs is honest. When they are all dishonest, they may always drop a ballot, hence recorded-as-intended cannot be guaranteed. To overcome this limitation, it would be necessary to introduce a public ballot board, which is not currently foreseen in the Swiss context.

## 5 Discussion

We have proposed a novel protocol that allows to split the trust assumptions during the setup between the Setup Component typically operated by the Swiss Cantons and the Control Components typically operated by a company (Swiss Post). Our protocol satisfies the double constraints of leaving the voter experience unchanged and having the Setup Component offline during the voting phase, which is also an additional security guarantee. In our protocol, the only component that is always trusted is the Printer and its channel to the Voters. This comes from the fact that, for usability reasons, Voters may only receive their voting material at once, by post. We may imagine in the future that the Voters have access to more sophisticated online services (at least for authentication), which could open new possibilities to weaken the trust on the Printer.

We have conducted a symbolic security analysis thanks to

the ProVerif tool, that shows that our protocol satisfies all the Federal Chancellery requirements. As future work, a computational security proof should be provided, as also required by the Federal Chancellery [18]. Interestingly, our protocol now relies on a generic group. This not only allows to opt for more efficient implementations (such as elliptic curves) but it also allows to get rid of non standard cryptographic assumptions such as SGSP in the current Swiss Post protocol [30].

## A Ethical Considerations

We did not identify any ethical issues related to this work:

- No attack on any existing product was revealed, so there is no need of responsible disclosure.
- The experiments that were performed did not involve any human being.
- The computational costs required for the security analysis with ProVerif were small enough to have negligible environmental impact compared to *e.g.* travelling to the conference.
- The use of Internet voting could be controversial; since we anchor this work in the realm of the Swiss Federal regulations, we estimate that the associated risks are taken into account (and actually, as mentioned in our work, we sometimes go beyond the requirements).

## B Open Science

In Section 4, we provide a security analysis based on the ProVerif tool. This tool is free software and publicly available for download. All the files modelling our protocol and its security properties in ProVerif have been made available as an artifact [2]. They should be enough to reproduce the results claimed in Table 1.

## References


[1] Ben Adida. Helios: Web-based Open-Audit Voting. In *USENIX'08*, pages 335–348, 2008.

[2] Anonymous. Artifact: ProVerif files for the security analysis. Available at https://anonymous.4open.science/r/artifact_usenix-6D82/, 2025.

[3] Josh Benaloh, Michael Naehrig, and Olivier Pereira. REACTIVE: Rethinking effective approaches concerning trustees in verifiable elections. Cryptology ePrint Archive, Paper 2024/915, 2024.

[4] David Bernhard, Véronique Cortier, Pierrick Gaudry, Mathieu Turuani, and Bogdan Warinschi. Verifiability analysis of CHVote. Cryptology ePrint Archive, Report 2018/1052, 2018. https://eprint.iacr.org/2018/1052.

[5] Karthikeyan Bhargavan, Bruno Blanchet, and Nadim Kobeissi. Verified models and reference implementations for the TLS 1.3 standard candidate. In *S&P'17*, pages 483–502, 2017.

[6] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1:1–135, 2016.

[7] Bruno Blanchet, Martín Abadi, and Cédric Fournet. Automated verification of selected equivalences for security protocols. In *20th IEEE Symposium on Logic in Computer Science (LICS 2005)*, pages 331–340. IEEE, 2005.

[8] Bruno Blanchet, Vincent Cheval, and Véronique Cortier. ProVerif with Lemmas, Induction, Fast Subsumption, and Much More. In *Symposium on Security and Privacy (SP)*, 2022.

[9] Mikaël Bougon, Hervé Chabanne, Véronique Cortier, Alexandre Debant, Emmanuelle Dottax, Jannik Dreier, Pierrick Gaudry, and Mathieu Turuani. Themis: an On-Site Voting System with Systematic Cast-as-intended Verification and Partial Accountability. In *25th ACM Conference on Computer and Communications Security (CCS'22)*, Los Angeles, United States, 2022. ACM.

[10] Achim Brelle and Tomasz Truderung. Cast-as-intended mechanism with return codes based on PETs. In *Electronic Voting: Second International Joint Conference, E-Vote-ID 2017, Bregenz, Austria, October 24-27, 2017, Proceedings 2*, pages 264–279. Springer, 2017.

[11] Margot Catinaud, Caroline Fontaine, and Guillaume Scerri. Proving e-voting mixnets in the CCSA model: zero-knowledge proofs and rewinding. Working paper or preprint, February 2025.

[12] Véronique Cortier, Alexandre Debant, and Vincent Cheval. Election verifiability with ProVerif. In *36th IEEE Computer Security Foundations Symposium (CSF'23)*, Dubrovnik, Croatia, July 2023.

[13] Véronique Cortier, Alexandre Debant, and Pierrick Gaudry. Breaking verifiability and vote privacy in CHVote. In *Esorics 2025 - 30th European Symposium on Research in Computer Security*, 2025.

[14] Véronique Cortier, Alexandre Debant, and Florian Moser. Code voting: when simplicity meets security. In *Esorics 2024 - 29th European Symposium on Research in Computer Security*, Bydgoszcz, Poland, 2024.

[15] Chris Culnane, Aleksander Essex, Sarah Jamie Lewis, Olivier Pereira, and Vanessa Teague. Knights and knaves run elections: Internet voting and undetectable electoral fraud. *IEEE Secur. Priv.*, 17(4):62–70, 2019.

[16] Stéphanie Delaune, Steve Kremer, and Mark D. Ryan. Verifying privacy-type properties of electronic voting protocols. *Journal of Computer Security*, 17(4):435–487, 2009.

[17] Federal Chancellery. Redesign and relaunch of trials – Final report of the Steering Committee Vote électronique (SC VE). https://www.bk.admin.ch/bk/en/home/politische-rechte/e-voting/berichte-und-studien.html, November 2020.

[18] Federal Chancellery. Federal Chancellery Ordinance on Electronic Voting. https://www.fedlex.admin.ch/eli/cc/2022/336/en, July 2022.

[19] Federal Chancellery. E-voting Catalogue of Measures by the Confederation and Cantons, Approved by the Vote électronique Steering Committee (SC VE): 4 August 2023. https://www.bk.admin.ch/dam/bk/en/dokumente/pore/E_Voting/E-voting%20Catalogue%20of%20measures%20by%20the%20Confederation%20and%20cantons,%204%20August%202023.pdf.download.pdf, 2023.

[20] David Galindo, Sandra Guasch, and Jordi Puiggali. Neuchâtel's Cast-as-Intended Verification Mechanism. In *International Conference on E-Voting and Identity (VOTE-ID)*, pages 3–18. Springer, 2015.

[21] Rolf Haenni, Reto E Koenig, and Eric Dubuis. Private Internet Voting on Untrusted Voting Devices. In *International Conference on Financial Cryptography and Data Security*, 2023.

[22] Rolf Haenni, Reto E Koenig, Philipp Locher, and Eric Dubuis. CHVote Protocol Specification. *Cryptology ePrint Archive*, 2017.

[23] Rolf Haenni, Philipp Locher, Reto Koenig, and Eric Dubuis. Pseudo-code algorithms for verifiable re-encryption mix-nets. In *Voting@FC*, pages 370–384. Springer, 2017. https://www.ifca.ai/fc17/voting/papers/voting17_HLKD17.pdf.

[24] Sven Heiberg and Jan Willemson. Verifiable internet voting in Estonia. In *International Conference on Electronic Voting (EVOTE)*, 2014.

[25] Nadim Kobeissi, Karthikeyan Bhargavan, and Bruno Blanchet. Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach. In *EuroS&P'17*, pages 435–450. IEEE, 2017.

[26] Moses D Liskov, John D Ramsdell, Joshua D Guttman, and Paul D Rowe. *The Cryptographic Protocol Shapes Analyzer: A Manual*. The MITRE Corporation, 2016.

[27] Ueli Maurer. Unifying zero-knowledge proofs of knowledge. In Bart Preneel, editor, *AFRICACRYPT 2009*, pages 272–286. Springer, 2009.

[28] Eleanor McMurtry, Olivier Pereira, and Vanessa Teague. When is a test not a proof? In *Computer Security – ESORICS 2020*, pages 23–41. Springer, 2020.

[29] Benedikt Schmidt, Simon Meier, Cas Cremers, and David Basin. Automated analysis of Diffie-Hellman protocols and advanced security properties. In *25th IEEE Computer Security Foundations Symposium (CSF'12)*, pages 78–94, 2012.

[30] Swiss Post. Protocol of the Swiss Post Voting System: Computational Proof of Complete Verifiability and Privacy. Version 1.4.0. Technical report, Swiss Post, 2025.

[31] Swiss Post. Swiss Post Voting System: Symbolic Analysis of the Swiss Post Voting System. Version 1.4.0. Technical report, Swiss Post, 2025.

[32] Swiss Post. Swiss Post Voting System: System Specification. Version 1.5.0. Technical report, Swiss Post, 2025.

[33] Björn Terelius and Douglas Wikström. Proofs of restricted shuffles. In *Progress in Cryptology – AFRICACRYPT 2010*, pages 100–113. Springer, 2010.

# C Non-classical zero-knowledge arguments

## C.1 Maurer's framework

In [27], Maurer describes a framework that covers many zero-knowledge proofs. The (slightly simplified) setting is as follows. Let $G_1$ and $G_2$ be groups that are finite products of cyclic groups of order $q$ where $q$ is a prime. Let $\phi : G_1 \to G_2$ be a group homomorphism that is efficiently computable by all parties. Let $z$ be a public element of $G_2$. The purpose of the protocol is to let the prover prove in zero-knowledge that they know an element $x$ in $G_1$ such that $\phi(x) = z$. We describe the interactive version, which can be turned into a non-interactive proof via the Fiat-Shamir transform.

The prover picks $k$, a uniform random element of $G_1$, computes $t = \phi(k)$, and sends $t$ to the verifier. Then, the verifier sends a uniform random element $c$ in $\mathbb{Z}_q$. The prover computes a response $r = k x^c$, and sends it to the verifier. The verifier computes $\phi(r)$ and $t\, z^c$, and accepts the proof if and only if they are equal.

We refer to the original paper for the security proof.

In the non-interactive version, where $c$ is obtained by hashing $t$ and the context, the prover can publish only $c$ and $r$, because $t$ can be recomputed by the verifier as $t = \phi(r)/z^c$, and then it has to check that indeed $c$ is obtained by hashing this value.

In terms of cost of the protocol, this amounts to one $\phi$-computation and one exponentiation in $G_1$ for the prover, and one $\phi$-computation and one exponentiation in $G_2$ for the verifier (the rest is considered to be negligible).

## C.2 $\mathsf{ZKP}_{\mathsf{ptxte}}$: Same plaintext, exponentiated

The $\mathsf{ZKP}_{\mathsf{ptxte}}$ zero-knowledge argument is used during the voting phase by the Client to prove that they encrypted the same vote $v$ in the part that goes to the tally and in the part that is used by the CC to select the appropriate return code.

The public data is

- the group $\mathbb{G}$, with generator $g$; and a second independent generator $h$;
- $\mathsf{pk}$, a public key;
- $C = (g^{r_1}, x\,\mathsf{pk}^{r_1})$, an encryption of $x$ with randomness $r_1$;
- $K = g^k$, a commitment to an exponent $k$;
- $y = x^k$, the plaintext $x$, raised to the power $k$.

The prover proves that they know $x$, $k$, $r_1$ such that $C$, $K$ and $y$ correspond to these values.

Since $(x,k) \mapsto x^k$ is not a group homomorphism, Maurer's framework does not directly apply. Therefore the prover will publish additional information and proceed in two steps. In the first step, they prove knowledge of $k$, and in the second step, they prove knowledge of $x$. The algebraic relations implied by these proofs will allow to deduce that $y = x^k$.

Additional data published by the prover:

- $\tilde{x} = x\, g^{r_2}$, where $r_2$ is a fresh, uniform random in $\mathbb{Z}_q$;
- $\tilde{h} = g^{r_3}$, where $r_3$ is a fresh, uniform random in $\mathbb{Z}_q$;
- $\tilde{y} = \tilde{x}^k\, h^{r_3}$.

The element $\tilde{x}$ is uniform random, and therefore leaks no information about $x$. The pair $(\tilde{h}, \tilde{y})$ is an ElGamal encryption, with a public key for which no-one knows the secret key. Therefore, under DDH, this does not leak any information about $\tilde{x}^k$.

**Step 1**: Prover proves that they know $k$ and $r_3$ such that $K = g^k$, $\tilde{y} = \tilde{x}^k h^{r_3}$, $\tilde{h} = g^{r_3}$.

The map $\phi_1 : \mathbb{Z}_q^2 \to G^3$ defined by

$$(\kappa, \rho) \mapsto (g^\kappa, \tilde{x}^\kappa h^\rho, g^\rho),$$

is publicly computable and is a group homomorphism, with $\phi_1(k, r_3) = (K, \tilde{y}, \tilde{h})$. We are therefore in the context of Maurer's framework, and the prover can prove in zero-knowledge that they know a pre-image of $(K, \tilde{y}, \tilde{h})$.

**Step 2**: Prover proves that they know $x$, $r_1$, $r_2$, $r_3$ such that $\tilde{h} = g^{r_3}$, $C = (g^{r_1}, x\,\mathsf{pk}^{r_1})$, $\tilde{x} = x\, g^{r_2}$, $\tilde{y}/y = K^{r_2} h^{r_3}$.

The map $\phi_2 : G \times \mathbb{Z}_q^3 \to G^5$ defined by

$$(\xi, \rho_1, \rho_2, \rho_3) \mapsto (\xi^{\rho_3}, g^{\rho_1}, \xi\,\mathsf{pk}^{\rho_1}, \xi\, g^{\rho_2}, K^{\rho_2} h^{\rho_3}),$$

is publicly computable and is a group homomorphism, with $\phi_2(x, r_1, r_2, r_3) = (\tilde{h}, C, \tilde{x}, \tilde{y}/y)$. The prover can prove in zero-knowledge that they know a pre-image of this (public) image.

**Soundness**: The extractor from the first step gives a pair $(k, r_3)$, and the extractor from the second step gives a tuple $(x, r_1, r_2, r_3')$. These extractors are independent, so that $r_3$ could be different from $r_3'$. However, we have $\tilde{h} = g^{r_3} = g^{r_3'}$, so that the exponent cannot differ. From here, we can compute $\tilde{y}$ in two different manners, implied by the two steps. This gives $\tilde{y} = \tilde{x}^k h^{r_3} = y K^{r_2} h^{r_3}$. The second step imposes that $\tilde{x} = x g^{r_2}$, so that finally, $y = x^k$, for the value of $x$ that is the plaintext associated to $C$ and the value of $k$ that is the discrete logarithm of $K$.

**Complexity**: The number of exponentiations for the prover is 15. Indeed, 4 exponentiations are required for computing the additional published data. Then $10 = 4 + 6$ exponentiations are needed for computing $\phi_1$ and $\phi_2$, and 1 further exponentiation occurs when doing an exponentiation in the domain of $\phi_2$.

For the verifier, the number of exponentiations amounts to 18. They come from 10 exponentiations for computing $\phi_1$ and $\phi_2$, and from 8 exponentiations for exponentiations in the co-domains of $\phi_1$ and $\phi_2$.

## C.3 $\mathsf{ZKP}_{\mathsf{SamePerm\&exp}}$: Same permutation, exponentiated

This ZKP is used in the setup phase, where a list of pairs of ciphertexts is submitted to a round of entities applying the DbleMixExp procedure, i.e. each entity will apply a shuffle to these pairs, and raise the second part of each pair to a (secret) exponent. We note that two different public keys are involved: one for the first ciphertexts of each pair, and one for the second ciphertexts.

The corresponding ZKP should ensure that the same permutation is used for both parts of the pairs, and the exponentiation is correct.

The verifiable mixnet of Terelius and Wikström [33] is versatile enough to allow this at a moderate cost. The "same permutation" part is essentially built-in, due to the structure of the ZKP. Inserting the exponentiation in the right place implies a few changes in the ZKP, but this combines without problem with the security proof.

For completeness, we give the full construction. Readers who are familiar with the classical construction can quickly go to Subsections C.3.3 and C.3.4. We use the notation of [23].

### C.3.1 Step 1: committing a permutation

Let $N$ be the number of ciphertexts. Let $g$, $h$, $h_1$, …, $h_N$ be (provably) independent generators. For a vector $\vec{v} = (v_1, \ldots, v_N) \in \mathbb{Z}_q^N$ and a random $\rho \in \mathbb{Z}_q$ we have that

$$\mathsf{Com}(\vec{v}, \rho) = g^{\rho} \prod_{1 \le i \le N} h_i^{v_i}$$

is a commitment of the vector $\vec{v}$ with opening $\rho$.

**Public/secret data** associated to a permutation $\sigma$.

Let $M_\sigma = (m_{ij})_{1 \le i,j \le N}$ be the matrix of $\sigma$: we have $m_{ij} = 1$ if $\sigma(i) = j$ and 0 otherwise.

Let $\vec{c} = (c_1, \ldots, c_N)$ where $c_i = \mathsf{Com}(i\text{-th col of } M_\sigma, r_i)$, a vector of commitments to columns of $M_\sigma$.

Let $\vec{u} = (u_1, \ldots, u_N) \in \mathbb{Z}_q^N$, obtained by hashing the full context (including all ciphertexts before and after shuffle) and $\vec{c}$.

The public data is $(\vec{c}, \vec{u})$.

The associated secret data is $(\sigma, M_\sigma, \vec{r} = (r_1, \ldots, r_N))$.

**Fact 1.** From [33] and [23], given the public data, proving (in ZK) the existence of $(\bar{r}, \tilde{r}, \vec{u}' = (u'_1, \ldots, u'_N)) \in \mathbb{Z}_q \times \mathbb{Z}_q \times \mathbb{Z}_q^N$ such that

(i) $\prod_{1 \le i \le N} c_i = \mathsf{Com}(\vec{1}, \bar{r}) = g^{\bar{r}} \prod_{1 \le i \le N} h_i$;

(ii) $\prod_{1 \le i \le N} u_i = \prod_{1 \le i \le N} u'_i$;

(iii) $\prod_{1 \le i \le N} c_i^{u_i} = \mathsf{Com}(\vec{u}', \tilde{r}) = g^{\tilde{r}} \prod_{1 \le i \le N} h_i^{u'_i}$;

proves that there exists a permutation $\sigma$ and a vector $\vec{r}$ with $c_i = \mathsf{Com}(M_{\sigma,i}, r_i)$ and $\vec{u}' = M_\sigma \vec{u}$.

### C.3.2 Step 2: an equivalent statement for (ii).

The statement (ii) in Fact 1. is not easily amenable to a ZKP. It can be converted to a sequence of chained commitments.

**Fact 2.** We keep the notations of the previous Fact. Let $\vec{\hat{c}} = (\hat{c}_1, \ldots, \hat{c}_N)$ be additional public values, and $\hat{c}_0 = h$ (for convenience of notation). Then, proving (in ZK) the existence of $\vec{\hat{r}} = (\hat{r}_1, \ldots, \hat{r}_N) \in \mathbb{Z}_q^N$ and $\hat{r} \in \mathbb{Z}_q$ such that

(i') For all $1 \le i \le N$, $\hat{c}_i = g^{\hat{r}_i} \hat{c}_{i-1}^{u'_i}$;

(ii') $\hat{c}_N = g^{\hat{r}} \, h^{(\prod_{1 \le i \le N} u_i)}$;

proves that $\prod_{1 \le i \le N} u_i = \prod_{1 \le i \le N} u'_i$.

**Sketch of proof.**
By (i'), and by induction, the prover knows the expression of each $\hat{c}_i$ as a combination of $g$ and $h$. And by (ii'), the prover knows the discret log of $\hat{c}_N / h^{(\prod_{1 \le i \le N} u_i)}$.
Therefore, by combining all equations of (i'), the prover knows an expression of $\hat{c}_N$ of the form $g^z h^{(\prod_{1 \le i \le N} u'_i)}$, for a known value of $z$. Equating those two expressions for $\hat{c}_N$, if $\prod u_i \neq \prod u'_i$, then the prover would deduce the discrete log of $h$ in base $g$, which is assumed to be impossible.
Furthermore, the sequence $(\hat{c}_1, \ldots, \hat{c}_N)$ is information theoretically hiding the $u'_i$.

### C.3.3 Step 3: Mixing and exponentiating at the same time.

From here, we diverge from [23]. We propose an NIZKP statement that is slightly different from [23, Equation (6)] to include an exponentiation.

We still assume that $g$, $h$, $h_1$, …, $h_N$ are independent generators. The following combines the statement of Fact 1, where (ii) is replaced by (i') and (ii') of Fact 2, and then the committed permutation is related to two vectors of ciphertexts.

**Proposition 1** *Let us consider the following public data:*

- $\vec{c} = (c_1, \ldots, c_N)$ *in* $\mathbb{G}^N$;
- $\vec{\hat{c}} = (\hat{c}_1, \ldots, \hat{c}_N)$ *in* $\mathbb{G}^N$, *and* $\hat{c}_0 = h$;
- $\vec{e} = (e_1, \ldots, e_N)$ *in* $(\mathbb{G} \times \mathbb{G})^N$, *ElGamal ciphertexts for public key* $\mathsf{pk}$;
- $\vec{e}' = (e'_1, \ldots, e'_N)$ *in* $(\mathbb{G} \times \mathbb{G})^N$, *other ElGamal ciphertexts;*
- $K$ *in* $\mathbb{G}$.

*Let* $\vec{u} = (u_1, \ldots, u_N)$ *in* $\mathbb{Z}_q^N$ *obtained by hashing all this public data.*

*Assume that the prover shows (in ZK) that there exists the following (secret) data:*

- $(k, \bar{r}, \hat{r}, \tilde{r}, r')$ *in* $\mathbb{Z}_q^5$;
- $\vec{u}' = (u'_1, \ldots, u'_N)$ *in* $\mathbb{Z}_q^N$;

- $\vec{\hat{r}} = (\hat{r}_1, \ldots, \hat{r}_N)$ *in* $\mathbb{Z}_q^N$;

*such that the following holds:*

*(i)* $\prod_{1\le i\le N} c_i = g^{\bar{r}} \prod_{1\le i\le N} h_i$;

*(ii)* $\hat{c}_N = g^{\hat{r}}\, h^{(\prod_{1\le i\le N} u_i)}$;

*(iii) For all* $1 \le i \le N$, $\hat{c}_i = g^{\hat{r}_i} \hat{c}_{i-1}^{u'_i}$;

*(iv)* $\prod_{1\le i\le N} c_i^{u_i} = g^{\tilde{r}} \prod_{1\le i\le N} h_i^{u'_i}$;

*(v)* $\prod_{1\le i\le N} e_i'^{u'_i} = \mathsf{enc}_{\mathsf{pk}}^{r'}(1) \cdot \prod_{1\le i\le N} e_i^{u_i \cdot k}$;

*(vi)* $K = g^k$.

*Then, the multi-set of plaintexts of* $\vec{e}'$ *is the same as the multi-set of plaintexts of* $\vec{e}$*, raised to the power* $k$ *such that* $K = g^k$.

**Sketch of proof.**
By (i) – (iv) and Facts 1 and 2, the prover shows that there exists a permutation $\sigma$ and a vector $\vec{u}'$ such that $\vec{u}' = M_\sigma \vec{u}$.
In order to prove that (v) implies that $e'_i$ is a permuted (by $\sigma$ re-reandomization of $e_i$ raised to the power $k$, we proceed as in [11, Lemma B.3] and show that for a random vector $\vec{u}$ (and then $\vec{u}'$ fixed by $\vec{u}' = M_\sigma \vec{u}$), the probability that there exists $r'$ such that (v) holds is in $1/q$, unless $\vec{e}'$ is indeed as claimed.
By (vi), the secret $k$ involved in (v) is indeed such that $K = g^k$.

### C.3.4 Step 4: The associated morphism.

In order to apply Maurer's framework, we write a group morphism $\phi$ such that the image by $\phi$ of the secret data $x$ of Proposition 1 is a publicly computable value $z$, with the property that $\phi(x) = z$ implies the properties (i)-(vi) of the Proposition 1.

We write $e_i = (a_i, b_i)$ and $e'_i = (a'_i, b'_i)$ the two parts of the input and output ciphertexts.

Let $\phi$ be the group morphism from $\mathbb{Z}_q^5 \times \mathbb{Z}_q^N \times \mathbb{Z}_q^N$ to $\mathbb{G}^{N+6}$, that, taking as input

$$(x_1, x_2, x_3, x_4, x_5), (\xi_1, \ldots, \xi_N), (\zeta_1, \ldots, \zeta_N)$$

gives

$$g^{x_1}, g^{x_2}, g^{x_3}, g^{x_4} \cdot \prod h_i^{\xi_i}, g^{x_5} \cdot \left(\prod (a_i^{u_i})^{x_1}\right) \cdot \left(\prod a_i'^{\xi_i}\right)^{-1},$$

$$\mathsf{pk}^{x_5} \cdot \left(\prod (b_i^{u_i})^{x_1}\right) \cdot \left(\prod b_i'^{\xi_i}\right)^{-1}, g^{\zeta_1} \hat{c}_0^{\xi_1}, \ldots, g^{\zeta_N} \hat{c}_{N-1}^{\xi_N}.$$

It is (boring but) easy to check that

$$\phi((k, \bar{r}, \hat{r}, \tilde{r}, r'), \vec{u}', \vec{\hat{r}}) = z =$$
$$= \left(K, (\prod c_i)/(\prod h_i), \hat{c}_N / h^{\prod u_i}, \prod c_i^{u_i}, 1, 1, \hat{c}_1, \ldots, \hat{c}_N\right),$$

which is publicly computable.

### C.3.5 Step 5: Simultaneously mixing another list of ciphertexts.

In order to use the same secret permutation to mix another set of ciphertexts, Proposition 1 can be modified as follows:

- add the other lists of input and output ciphertexts to the public data;
- add the vector or random used for re-randomizing this other list to the secret data;
- add a condition (vii), similar to (v), without the exponent $k$.

Note that the public key involved in this other list of ciphertexts does not need to be the same as before.

### C.3.6 Complexity

The number of operations for the prover amounts to:

- Computing the $c_i$'s and the $\hat{c}_i$'s: cost = $3N$ Exp.
- Computing $\phi$ of a random element:
  - Components that relates only to the permutation: cost = $3N$ Exp.
  - Components that relates to the ciphertexts that are not exponentiated: $2N$ Exp.
  - Components that relates to the ciphertexts that are exponentiated: $4N$ Exp.
- Other parts that cost $O(1)$ Exp.

The total for the prover is therefore $12N + O(1)$ exponentiations in $\mathbb{G}$.

The number of operations for the verifier amounts to:

- Computing $\phi$ of the response:
  - Components that relates only to the permutation: cost = $4N$ Exp.
  - Components that relates to the ciphertexts that are not exponentiated: $4N$ Exp.
  - Components that relates to the ciphertexts that are exponentiated: $4N$ Exp.
- Computing one exponentiation in the image group: cost = $N + O(1)$ Exp.
- Other parts that cost $O(1)$ Exp.

The total for the verifier is therefore $13N + O(1)$ exponentiations in $\mathbb{G}$.

## D Complexity of each phase of the protocol

We recall the notation: $m$ is the number of CCs; $n$ is the number of candidates; $N$ is the number of Voters; $n_{\text{bal}}$ is the number of ballots that have been cast. We count the number of group exponentiations (Exp) and focus on the dominant terms.

### D.1 Setup phase

For each Voter, each component must encrypt its $\mathsf{pRC}_{j,i}$ share for the Printer with a ZKP ($3n$ Exp), verify these ZKPs of others ($2mn$ Exp), apply DbleMixExp ($17n$ Exp), and verify the associated ZKPs of others ($13mn$ Exp). The rest is negligible. The SC does an additional encryption of its $\mathsf{pRC}_{0,i}$ share with $\mathsf{pk}_{\mathsf{id}}$ ($2n$ Exp) and a ZKP ($n$ Exp), that is verified by each CC ($2n$ Exp).

The total cost is therefore $(15m+22)n$ Exp for each CC, and $(15m+23)n$ Exp for SC. Note that the dominant $(15mn)$ part can be externalized to Auditors. This can also be delayed, but must be done before the voting phase starts.

### D.2 Voting phase

For each Voter, the cost for the online CC's is small: the only part that is not in $O(1)$ Exp is the PET protocol. Since there is a single PET, this means $O(m)$ exponentiations. We consider this to be negligible and do not make the constants explicit.

For the Client, the computing power is limited, so we are more precise. It needs to perform 18 Exp for the first communication: 2 for the encryption, 1 for masking, and 15 for the ZKP. For the second communication, there is 2 Exp for the encryption, 1 Exp for the ZKP, and 1 Exp for the signature. In total, this gives 22 Exp. In addition, the Client must also establish $m$ secure channels.

### D.3 Tally phase

We assume that there are $m+1$ Talliers (typically SC and all the CCs). Based on the pseudo-code of [23], we get the following costs. Computing a verifiable shuffle costs $10n_{\text{bal}}$ Exp, and verifying the $m$ shuffles of the others costs $8mn_{\text{bal}}$ Exp. The partial decryption (and its ZKP) costs $3n_{\text{bal}}$ exponentiations and verifying them costs $4mn_{\text{bal}}$ exponentiations. The total cost for each Tallier is therefore $(12m+13)n_{\text{bal}}$ Exp. Again the dominant $12mn_{\text{bal}}$ part can be externalized and delayed, but the ZKPs of the shuffles must be verified before decryption.